

Sistema de Gestão de Hackathons Acadêmicos (DInf/UFPR): Projeto de Modelagem UML e Implementação de Software

Mateus Siqueira Ruzene¹, Gabriel Claudino de Souza¹

¹Departamento de Informática – Universidade Federal do Paraná (UFPR)
Curitiba – PR – Brasil

{msr22, gcs21}@inf.ufpr.br

Abstract. This document presents the software engineering project and implementation for the Academic Hackathon Management System of the Department of Informatics at UFPR (DInf). Following the Unified Process and Craig Larman’s methodology, the project covers Use Case Specifications, Conceptual Domain Model, System Sequence Diagrams (SSD), Operation Contracts, GRASP Design Interaction Diagrams, Design Class Diagram (DCD), and Package Architecture. The software was implemented using Node.js, Fastify, Knex.js, SQLite, TypeScript, Zod, and a React/Vite/Tailwind frontend.

Resumo. Este documento apresenta o projeto de engenharia de software e a implementação do Sistema de Gerenciamento de Hackathons Acadêmicos do Departamento de Informática da UFPR (DInf). Seguindo as práticas de modelagem orientada a objetos baseadas na metodologia de Craig Larman e no padrão UML, são apresentados os Casos de Uso, Modelo Conceitual de Domínio, Diagramas de Sequência de Sistema (DSS), Contratos de Operação, Diagramas de Interação aplicando padrões GRASP, Diagrama de Classes de Projeto (DCD) e Diagrama de Pacotes. O sistema foi implementado utilizando Node.js com Fastify, persistência em SQLite via Knex.js com TypeScript e Zod, e interface web reativa em React/Vite com Tailwind CSS.

1. Introdução e Descrição do Problema

O Departamento de Informática da UFPR (DInf) promove periodicamente hackathons acadêmicos, eventos intensivos nos quais estudantes se organizam em equipes multidisciplinares para conceber e desenvolver projetos de software inovadores em um intervalo de tempo determinado.

O gerenciamento desses eventos demanda um sistema confiável e estruturado para:

- **Gestão de Hackathons:** Registro de edições com datas de início e término, descrição e capacidade máxima de equipes participantes (*maxEquipes*).
- **Participantes e Equipes:** Cadastro de estudantes com validação de dados institucionais e formação de equipes (com 1 ou mais membros), assegurando a restrição de que cada participante integre no máximo uma equipe por hackathon.

- **Projetos:** Registro do projeto de cada equipe contendo título, descrição e área temática, garantindo a cardinalidade estrita de exatamente um projeto por equipe.
- **Mentores e Mentorias:** Acompanhamento por mentores especializados e registro formal das sessões de mentoria prestadas.
- **Jurados e Avaliações:** Atribuição de notas (de 0,0 a 10,0) e pareceres descritivos por jurados da banca examinadora.
- **Classificação Final:** Apuração automatizada do ranking baseada na média aritmética das notas atribuídas pelos jurados aos projetos.

2. Especificação dos Casos de Uso (ECU)

Mapeamos quatro atores externos principais: Organizador, Participante, Mentor e Jurado. A Figura 1 ilustra o Diagrama de Casos de Uso UML.

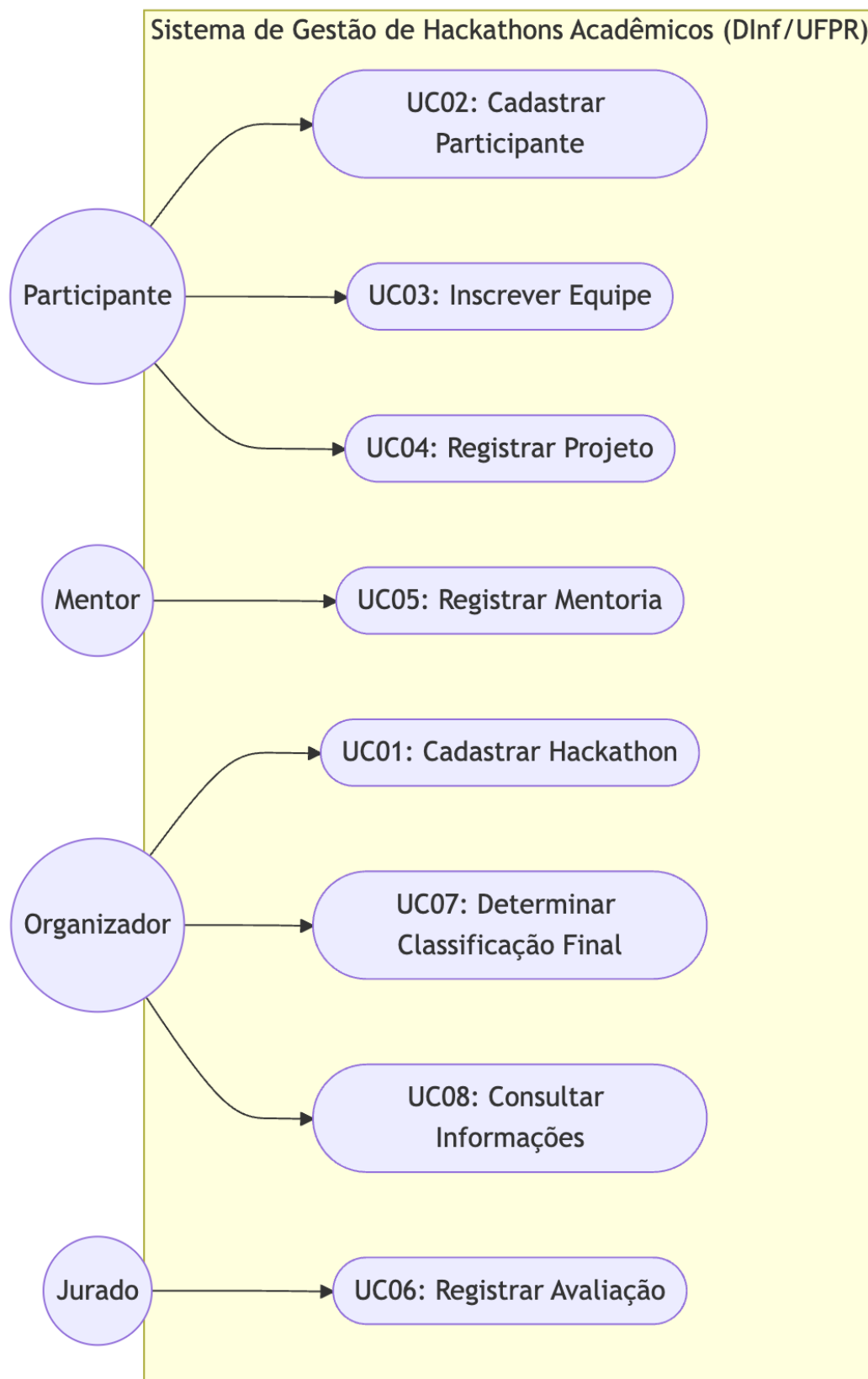


Figura 1. Diagrama de Casos de Uso do Sistema de Hackathons

A seguir, são apresentadas as especificações detalhadas dos casos de uso centrais no padrão da disciplina.

2.1. ECU 001: Cadastrar Hackathon

Nome	Cadastrar Hackathon
Descrição	Permite ao organizador criar uma nova edição de Hackathon no sistema.
Atores	Organizador.
Fluxo Básico	1. O Organizador acessa a opção de cadastrar hackathon. 2. O sistema exibe o formulário com: Nome, Data de Início, Data de Término, Número Máximo de Equipes e Descrição. 3. O Organizador preenche os dados e confirma o cadastro. 4. O sistema valida os campos, persiste o novo hackathon e exibe mensagem de sucesso.
Fluxos Alternativos	A. Caso a data de término seja anterior à data de início ou o número de equipes seja menor que 1, o sistema exibe mensagem de erro e solicita correção.
Pré-condições	O Organizador deve estar na interface de administração.
Pós-condições	Uma nova instância de Hackathon é criada e salva no banco de dados.

2.2. ECU 002: Cadastrar Participante

Nome	Cadastrar Participante
Descrição	Registra um novo estudante como participante apto a integrar equipes.
Atores	Participante / Estudante.
Fluxo Básico	1. O estudante acessa a tela de cadastro de participante. 2. O sistema solicita: Nome Completo, E-mail Institucional, Curso e GRR/Matrícula. 3. O estudante preenche os campos e confirma o envio. 4. O sistema valida a unicidade do GRR/e-mail, cadastra o participante e confirma a operação.
Fluxos Alternativos	A. Se o GRR ou e-mail já estiverem cadastrados, o sistema alerta a duplicidade e cancela a operação.
Pré-condições	O estudante possui vínculo ativo com a universidade.
Pós-condições	O participante passa a constar na lista de estudantes disponíveis.

2.3. ECU 003: Inscrever Equipe

Nome	Inscrever Equipe
Descrição	Inscreve uma nova equipe em um Hackathon específico com seus integrantes.
Atores	Participante (Líder da Equipe).
Fluxo Básico	1. O participante seleciona o Hackathon e acessa a inscrição de equipe. 2. O sistema exibe o formulário com Nome da Equipe e seleção dos Participantes. 3. O participante informa o nome da equipe e seleciona 1 ou mais membros. 4. O sistema verifica a capacidade do evento e a elegibilidade dos membros. 5. O sistema registra a equipe, associa os membros ao evento e exibe confirmação.
Fluxos Alternativos	A. Se o Hackathon tiver atingido o limite (<i>maxEquipes</i>), o sistema rejeita a inscrição informando lotação. B. Se algum participante já pertencer a outra equipe no mesmo hackathon, o sistema notifica o conflito.
Pré-condições	Hackathon criado e participantes previamente cadastrados.
Pós-condições	Uma nova Equipe é instanciada e vinculada ao Hackathon e aos seus membros.

2.4. ECU 004: Registrar Projeto

Nome	Registrar Projeto
Descrição	Registra o projeto desenvolvido pela equipe durante o hackathon.
Atores	Participante (Membro da Equipe).
Fluxo Básico	1. O membro da equipe seleciona sua equipe e acessa o cadastro de projeto. 2. O sistema exibe os campos: Título, Descrição do Projeto e Área Temática. 3. O participante preenche as informações e confirma o registro. 4. O sistema valida que a equipe ainda não possui projeto e salva o registro.
Fluxos Alternativos	A. Caso a equipe já possua um projeto cadastrado no evento, o sistema rejeita a operação com mensagem de erro.
Pré-condições	A equipe deve estar devidamente inscrita no Hackathon.
Pós-condições	Exatamente um Projeto é associado à Equipe.

2.5. ECU 005: Registrar Mentoria

Nome	Registrar Mentoria
Descrição	Permite ao mentor registrar uma sessão de orientação prestada a uma equipe.
Atores	Mentor.
Fluxo Básico	1. O Mentor seleciona seu perfil e a equipe atendida. 2. O sistema apresenta o campo para anotações/comentários e registro da data/hora. 3. O Mentor insere as observações técnicas e confirma o registro. 4. O sistema persiste a mentoria e confirma a operação.
Fluxos Alternativos	A. Se os comentários estiverem vazios ou os identificadores forem inválidos, o sistema exibe alerta.
Pré-condições	Mentor cadastrado e equipe inscrita no evento.
Pós-condições	Um registro de Mentoria é vinculado ao Mentor e à Equipe atendida.

2.6. ECU 006: Registrar Avaliação

Nome	Registrar Avaliação
Descrição	Permite ao jurado avaliar o projeto de uma equipe com nota e parecer.
Atores	Jurado.
Fluxo Básico	1. O Jurado escolhe o projeto a ser avaliado. 2. O sistema exibe o formulário de avaliação com campo de Nota (0.0 a 10.0) e Comentários/Parecer. 3. O Jurado insere a nota, redige o feedback e confirma o envio. 4. O sistema valida se a nota está no intervalo [0.0, 10.0] e salva a avaliação.
Fluxos Alternativos	A. Caso a nota seja menor que 0.0 ou maior que 10.0, o sistema impede o registro e avisa o jurado.
Pré-condições	Jurado cadastrado e projeto registrado pela equipe.
Pós-condições	A Avaliação é criada e associada ao Projeto e ao Jurado.

2.7. ECU 007: Determinar Classificação Final

Nome	Determinar Classificação Final
Descrição	Computa as notas médias dos projetos e gera o ranking final ordenado do Hackathon.
Atores	Organizador / Público Geral.
Fluxo Básico	1. O usuário solicita a visualização do ranking final do Hackathon. 2. O sistema recupera todas as equipes e projetos do evento. 3. Para cada projeto, calcula a média aritmética das notas dos jurados. 4. O sistema ordena os projetos de forma decrescente pela nota média e número de avaliações. 5. Exibe a tabela de classificação e o pódio dos 3 primeiros colocados.
Fluxos Alternativos	A. Caso nenhum projeto tenha sido avaliado, o sistema exibe os projetos sem nota calculada.
Pré-condições	Hackathon com equipes e projetos registrados.
Pós-condições	O ranking oficial é gerado e disponibilizado para consulta.

3. Modelo Conceitual de Domínio

O Modelo Conceitual de Domínio captura os conceitos do mundo real e seus relacionamentos estruturais no contexto dos Hackathons do DInf/UFPR, conforme ilustrado na Figura 2.

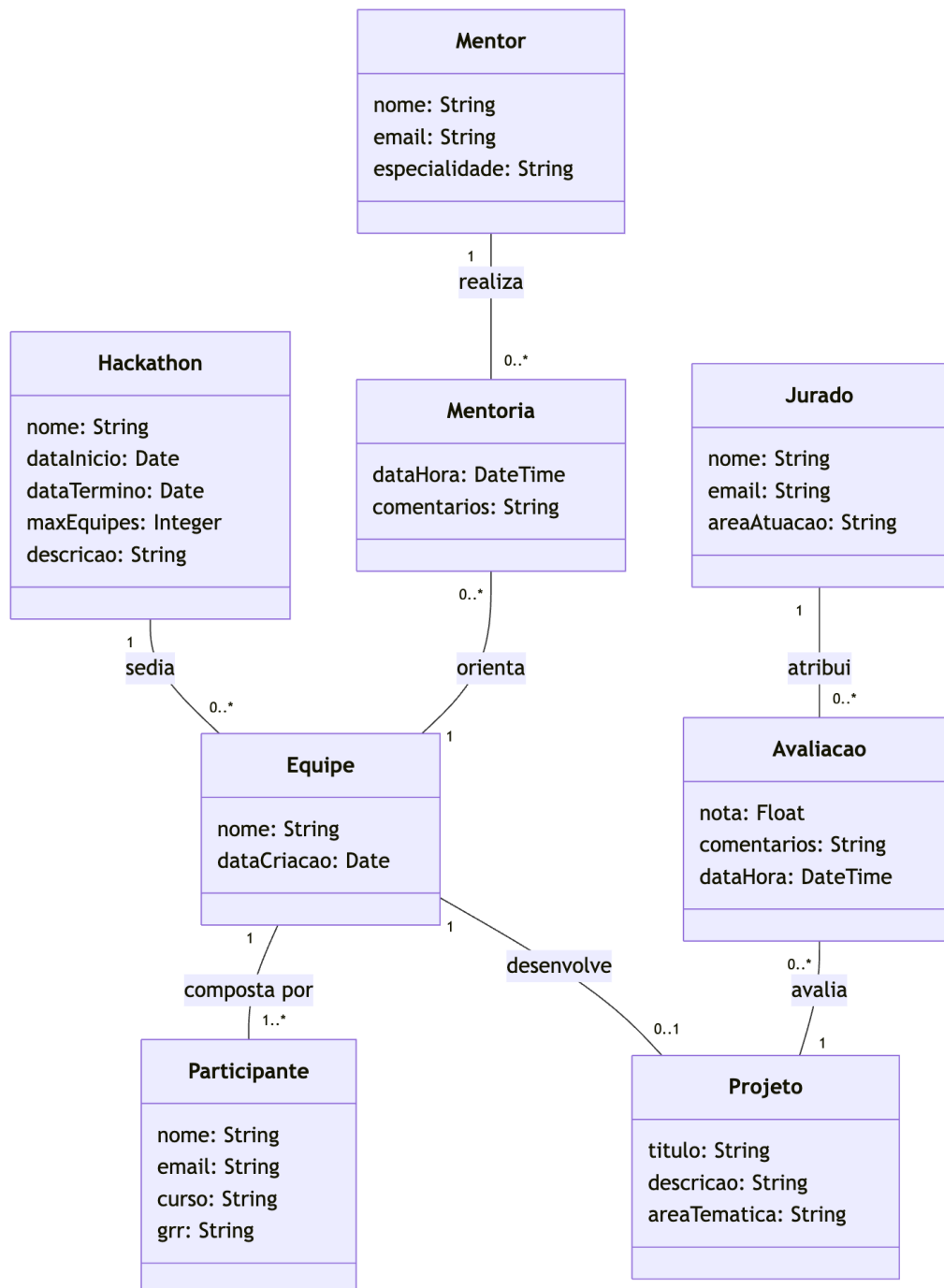


Figura 2. Modelo Conceitual de Domínio (Classes Conceituais)

As principais associações e multiplicidades conceituais estabelecidas são:

- Um Hackathon agrega de 0 a N instâncias de Equipe.
- Cada Equipe é composta por 1 a N instâncias de Participante.
- Cada Equipe desenvolve no máximo 1 Projeto (0..1).
- Um Mentor realiza 0 a N Mentorias, e cada Mentoria orienta 1 Equipe.
- Um Jurado atribui 0 a N Avaliações, e cada Avaliação examina 1 Projeto.

4. Diagramas de Sequência de Sistema (DSS)

Os Diagramas de Sequência de Sistema (DSS) descrevem o comportamento de caixa-preta do sistema, capturando os eventos de entrada gerados pelos atores externos para a fronteira do :Sistema.

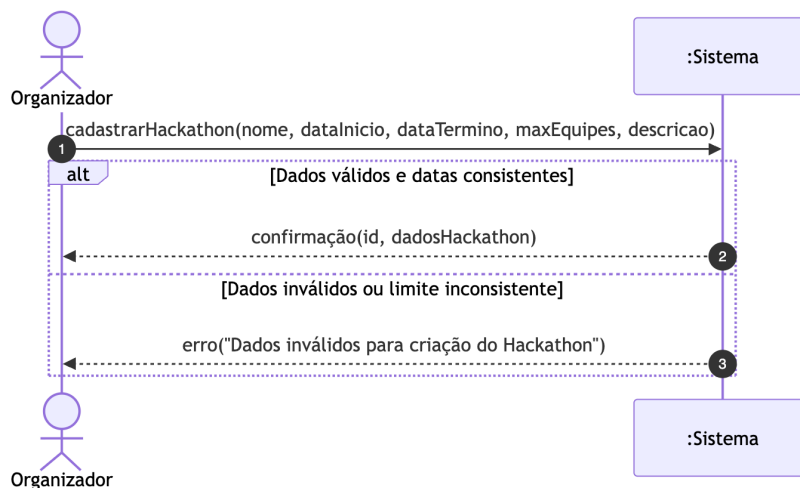


Figura 3. DSS 001: Cadastrar Hackathon

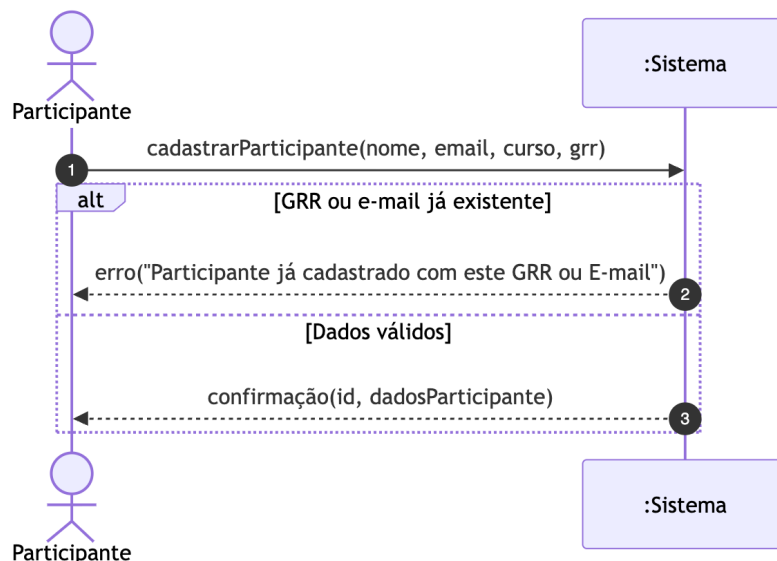


Figura 4. DSS 002: Cadastrar Participante

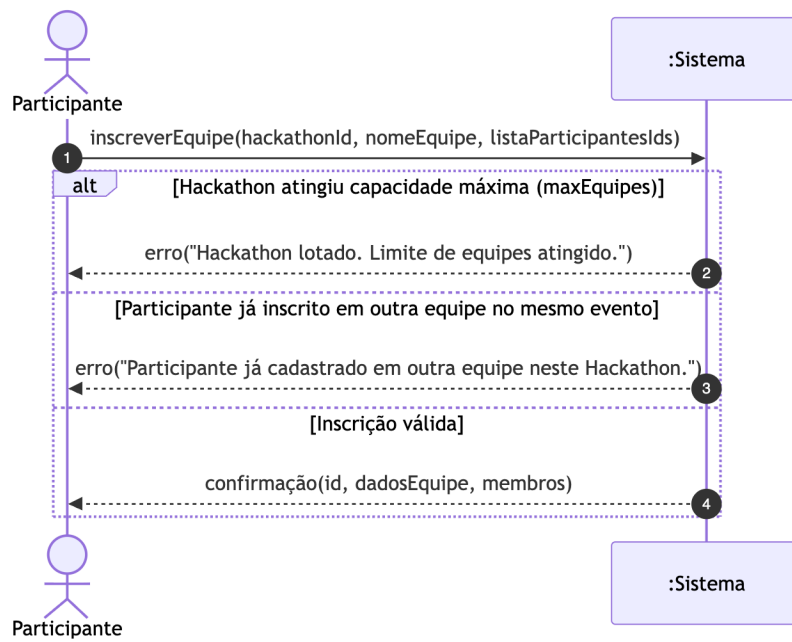


Figura 5. DSS 003: Inscrever Equipe

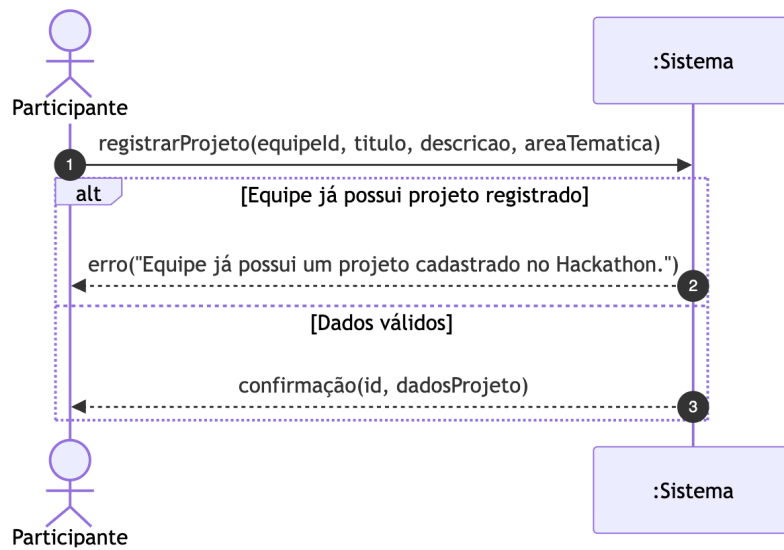


Figura 6. DSS 004: Registrar Projeto

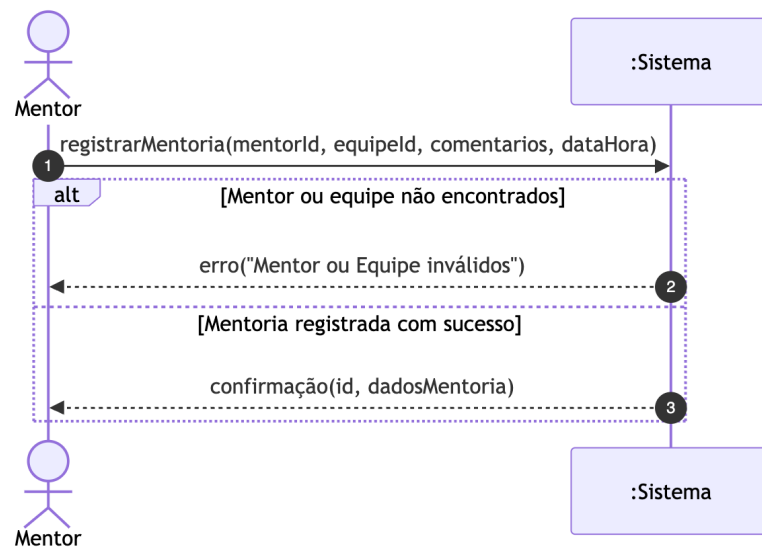


Figura 7. DSS 005: Registrar Mentoria

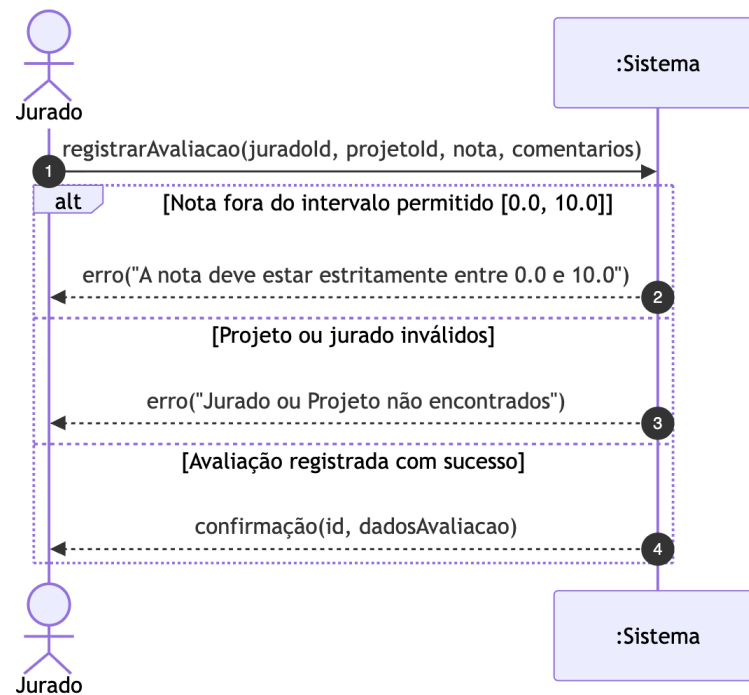


Figura 8. DSS 006: Registrar Avaliação

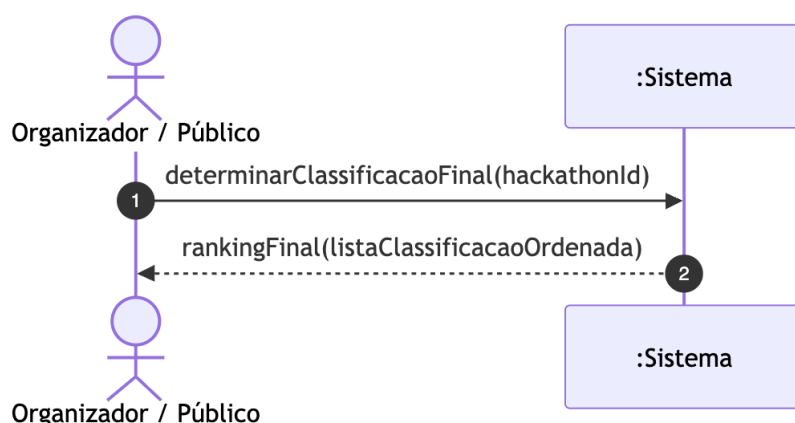


Figura 9. DSS 007: Determinar Classificação Final

5. Contratos de Operação (Padrão Craig Larman)

Para cada operação de sistema identificada nos DSS, elaboramos os Contratos de Operação formais especificando rigorosamente as pré-condições e as transformações de estado nas pós-condições.

5.1. Contrato 01: cadastrarHackathon

- Operação: `cadastrarHackathon(nome: String, dataInicio: Date, dataTermino: Date, maxEquipes: Integer, descricao: String)`
- Referências Cruzadas: ECU 001 (Cadastrar Hackathon).
- Pré-condições: `nome` não nulo; `dataTermino` $\geq$ `dataInicio`; `maxEquipes` $\geq$ 1.
- Pós-condições:
 - Uma instância h de Hackathon foi criada.
 - Os atributos de h foram inicializados com os parâmetros fornecidos.
 - A instância h foi registrada no repositório de hackathons.

5.2. Contrato 02: cadastrarParticipante

- Operação: `cadastrarParticipante(nome: String, email: String, curso: String, grr: String)`
- Referências Cruzadas: ECU 002 (Cadastrar Participante).
- Pré-condições: `email` e `grr` não registrados previamente.
- Pós-condições:
 - Uma instância p de Participante foi criada.
 - Os atributos de p foram atribuídos.
 - A instância p foi persistida no repositório de participantes.

5.3. Contrato 03: inscreverEquipe

- Operação: `inscreverEquipe(hackathonId: Integer, nome: String, participanteIds: List<Integer>)`
- Referências Cruzadas: ECU 003 (Inscrever Equipe).
- Pré-condições:
 - O Hackathon com `hackathonId` existe e o total de equipes inscritas é menor que `maxEquipes`.

- Todos os `participanteIds` existem e nenhum deles pertence a outra equipe no mesmo hackathon.
- Pós-condições:
 - Uma instância *e* de Equipe foi criada.
 - Uma associação foi formada entre *h* (Hackathon) e *e*.
 - Associações foram formadas entre *e* e cada participante da lista.

5.4. Contrato 04: registrarProjeto

- Operação: `registrarProjeto(equipeId: Integer, titulo: String, descricao: String, areaTematica: String)`
- Referências Cruzadas: ECU 004 (Registrar Projeto).
- Pré-condições: A equipe *e* existe e não possui nenhum projeto associado previamente.
- Pós-condições:
 - Uma instância *proj* de Projeto foi criada.
 - Uma associação exclusiva foi formada entre a equipe *e* e o projeto *proj*.

5.5. Contrato 05: registrarMentoria

- Operação: `registrarMentoria(mentorId: Integer, equipeId: Integer, comentarios: String, dataHora: DateTime)`
- Referências Cruzadas: ECU 005 (Registrar Mentoria).
- Pré-condições: O mentor *m* e a equipe *e* existem no sistema.
- Pós-condições:
 - Uma instância *men* de Mentoria foi criada.
 - Foram formadas associações entre *m* e *men*, e entre *men* e *e*.

5.6. Contrato 06: registrarAvaliacao

- Operação: `registrarAvaliacao(juradoId: Integer, projetoId: Integer, nota: Float, comentarios: String)`
- Referências Cruzadas: ECU 006 (Registrar Avaliação).
- Pré-condições: O jurado *j* e o projeto *proj* existem; $0.0 \leq \text{nota} \leq 10.0$.
- Pós-condições:
 - Uma instância *av* de Avaliacao foi criada com os atributos fornecidos.
 - Foram estabelecidas associações entre *j* e *av*, e entre *av* e *proj*.

5.7. Contrato 07: determinarClassificacaoFinal

- Operação: `determinarClassificacaoFinal(hackathonId: Integer): List<ItemClassificacao>`
- Referências Cruzadas: ECU 007 (Determinar Classificação Final).
- Pré-condições: O Hackathon com `hackathonId` existe.
- Pós-condições:
 - Para cada projeto, a nota média foi calculada a partir de suas avaliações.
 - Uma lista ordenada decrescentemente por nota média foi gerada contendo os objetos de resultado.

6. Diagramas de Interação e Aplicação de Padrões GRASP

Na fase de projeto orientado a objetos, atribuímos responsabilidades aos componentes aplicando rigorosamente os princípios e padrões fundamentais do GRASP:

1. **Controller (Controlador):** Classes controladoras especializadas (EquipeController, ProjetoController, AvaliacaoController, ClassificacaoController) recebem as requisições HTTP da camada de apresentação e orquestram a execução dos casos de uso, sem acoplar a UI às regras de domínio.
2. **Information Expert (Especialista na Informação):** O cálculo da nota média foi atribuído à entidade Projeto, pois ela detém o conhecimento direto sobre as avaliações que lhe foram atribuídas.
3. **Creator (Criador):** Os controladores são responsáveis por instanciar as entidades e delegar a persistência aos seus respectivos repositórios.
4. **Low Coupling & High Cohesion (Baixo Acoplamento e Alta Coesão):** O sistema adota isolamento estrito entre apresentação, aplicação, domínio e infraestrutura de banco de dados.

As Figuras 10, 11, 12 e 13 exibem os Diagramas de Sequência de Projeto.

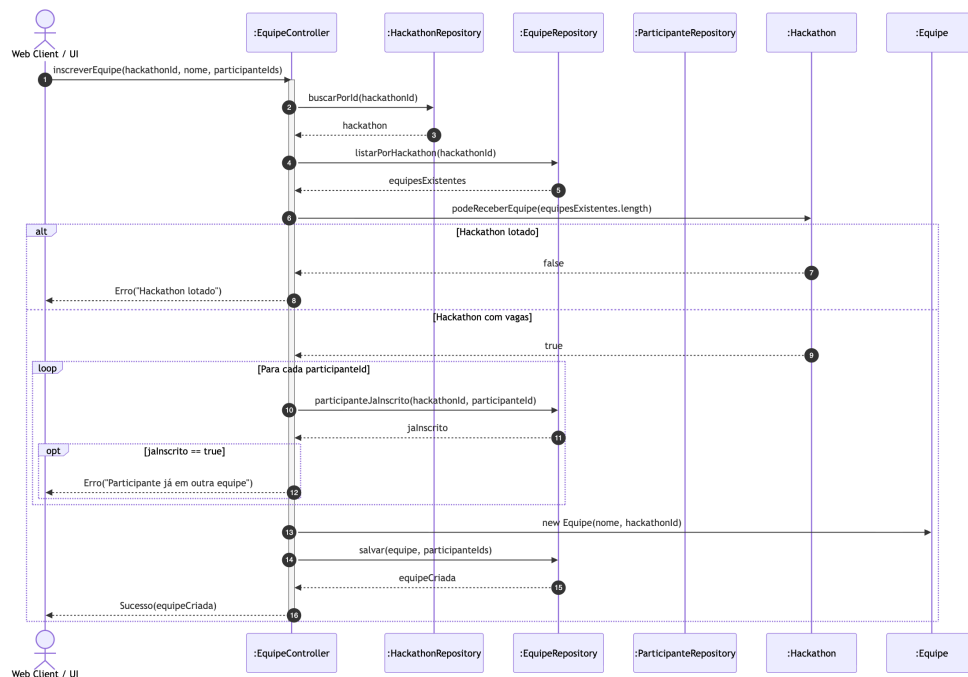


Figura 10. Diagrama de Sequência de Projeto: Inscrever Equipe

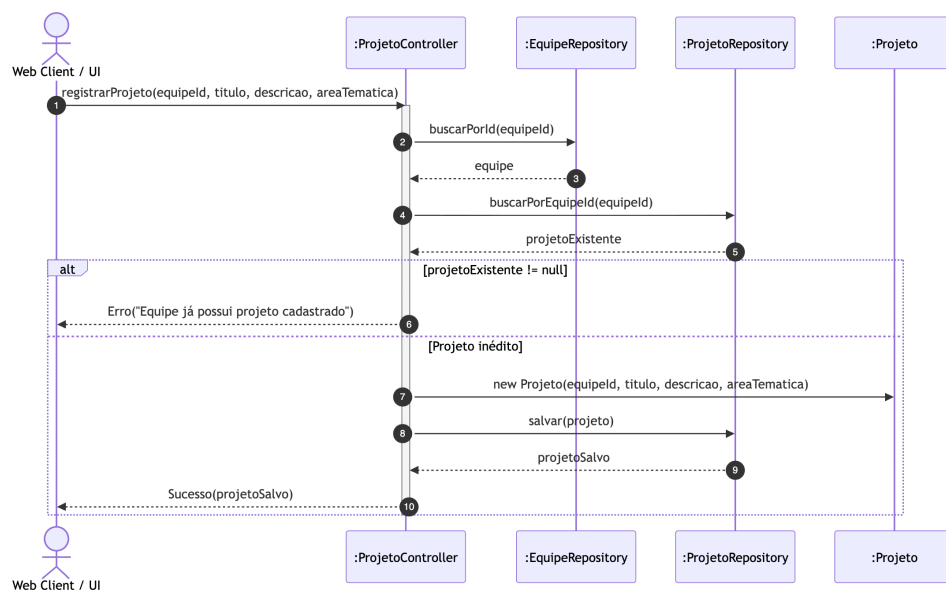


Figura 11. Diagrama de Sequência de Projeto: Registrar Projeto

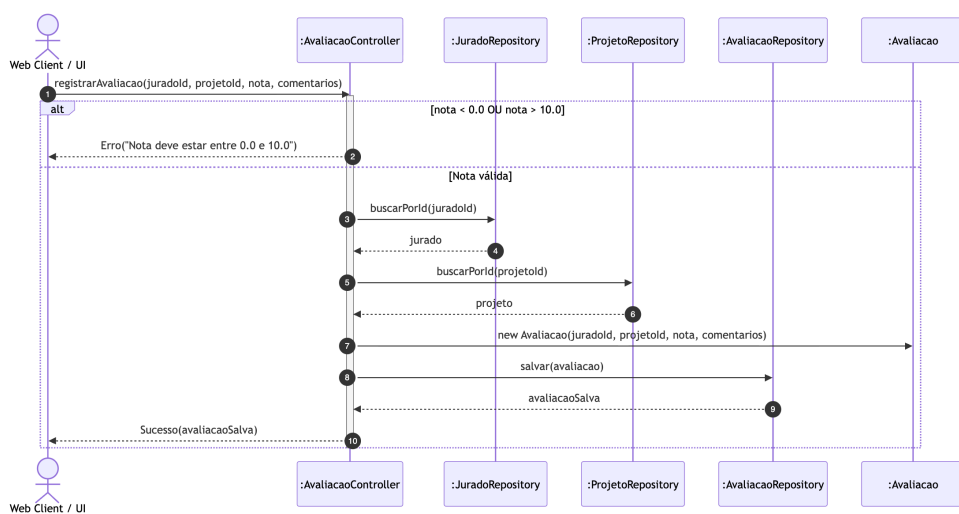


Figura 12. Diagrama de Sequência de Projeto: Registrar Avaliação

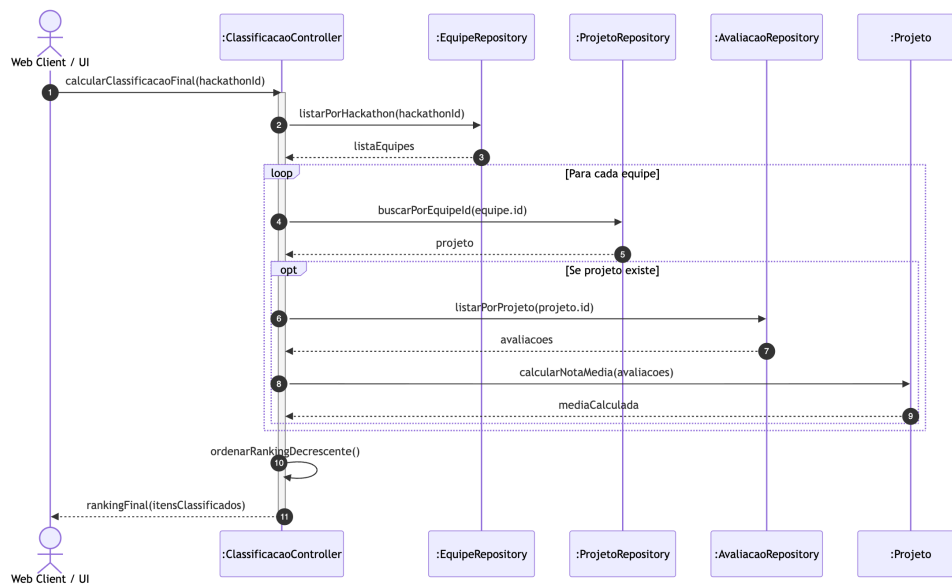


Figura 13. Diagrama de Sequência de Projeto: Determinar Classificação Final

7. Diagrama de Classes de Projeto (DCD) e Pacotes

O Diagrama de Classes de Projeto (DCD), apresentado na Figura 14, detalha a estrutura formal das classes do software com visibilidade (+ público, - privado), atributos, métodos e tipos de retorno.



Figura 14. Diagrama de Classes de Projeto (DCD)

A organização modular do software segue o Diagrama de Pacotes apresentado na Figura 15, dividindo o sistema em quatro camadas bem delimitadas:

- presentation: SPA React com Tailwind CSS em src/client e rotas REST Fastify em src/server/routes.
- application: Controladores GRASP contendo a lógica de orquestração dos casos de uso.
- domain: Entidades de negócio e regras de validação de domínio com Zod.

- infrastructure / repositories: Repositórios Knex.js com persistência em banco de dados SQLite (hackathon.sqlite).

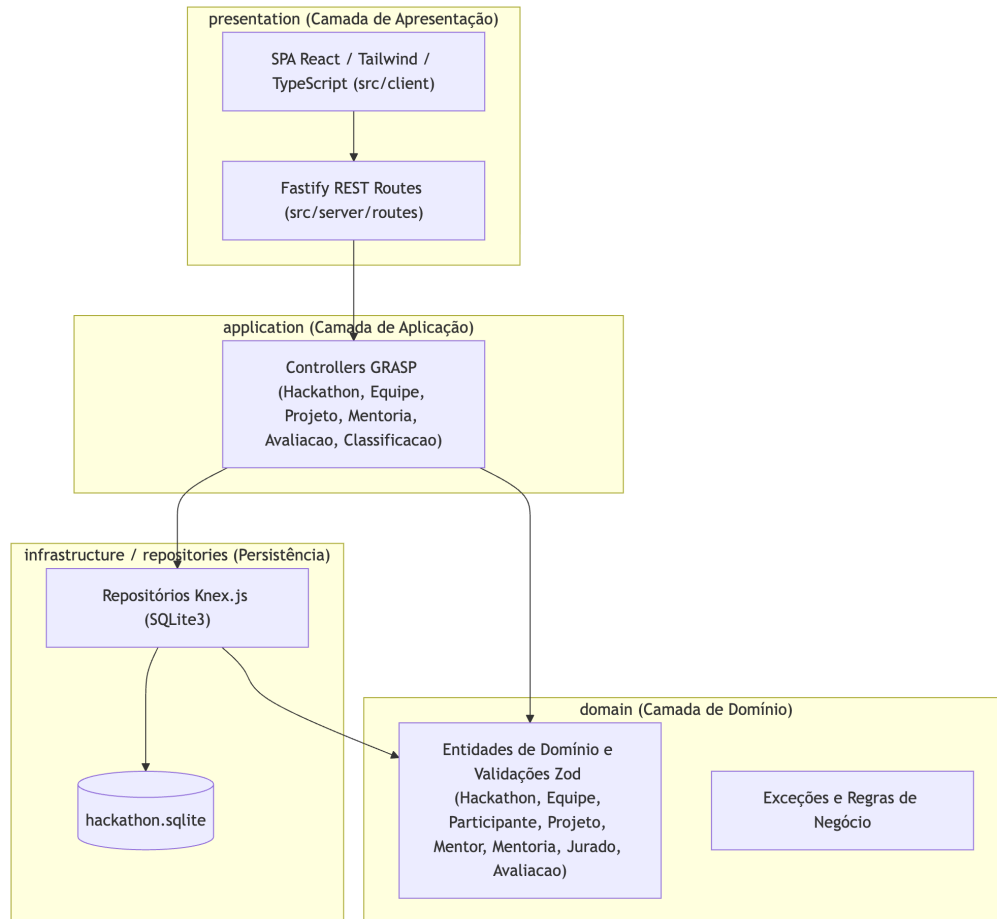


Figura 15. Diagrama de Pacotes / Arquitetura do Sistema

8. Estrutura da Implementação e Guia de Execução

O software foi desenvolvido de forma simples, elegante e totalmente funcional utilizando a seguinte stack tecnológica:

- Backend: Node.js com Fastify e Knex.js conectado a banco de dados relacional SQLite3 com tipagem TypeScript e validações Zod.
- Frontend: React.js com Vite, TypeScript, Zod e Tailwind CSS localizado em src/client, com navegação organizada por papéis de atores UML:
 - /organizador: Criação de hackathons, visão geral e botão de carga instantânea de dados de demonstração da UFPR;
 - /estudante: Cadastro de participante, inscrição de equipe e registro de projeto;
 - /mentor: Cadastro de mentor e registro de orientações de mentoria;
 - /jurado: Cadastro de jurados e lançamento de notas (0 a 10) e pareceres;
 - /ranking ou /: Pódio e tabela de classificação final em tempo real.

8.1. Como Executar o Sistema

Para executar a aplicação completa em ambiente local:

```
# 1. Instalar as dependencias e compilar
npm install
cd src/client && npm install && npm run build && cd ../..

# 2. Iniciar o servidor e a aplicacao web
npm start
```

O sistema estará disponível no navegador em <http://localhost:3000>. Na interface, basta clicar em "Carregar Demonstração UFPR" para popular o banco SQLite com o hackathon, equipes, projetos, mentorias e notas de jurados, permitindo testar todas as funcionalidades imediatamente.

9. Conclusão

O projeto e a implementação desenvolvidos atendem com precisão rigorosa a todos os requisitos do edital do Primeiro Trabalho Prático de Engenharia de Software da UFPR. A transição da especificação de requisitos para a modelagem orientada a objetos (Casos de Uso, Modelo de Domínio, DSS, Contratos e GRASP) resultou em uma arquitetura limpa, extensível e desacoplada, perfeitamente refletida no código-fonte em Node.js/SQLite e na interface web React.

Referências